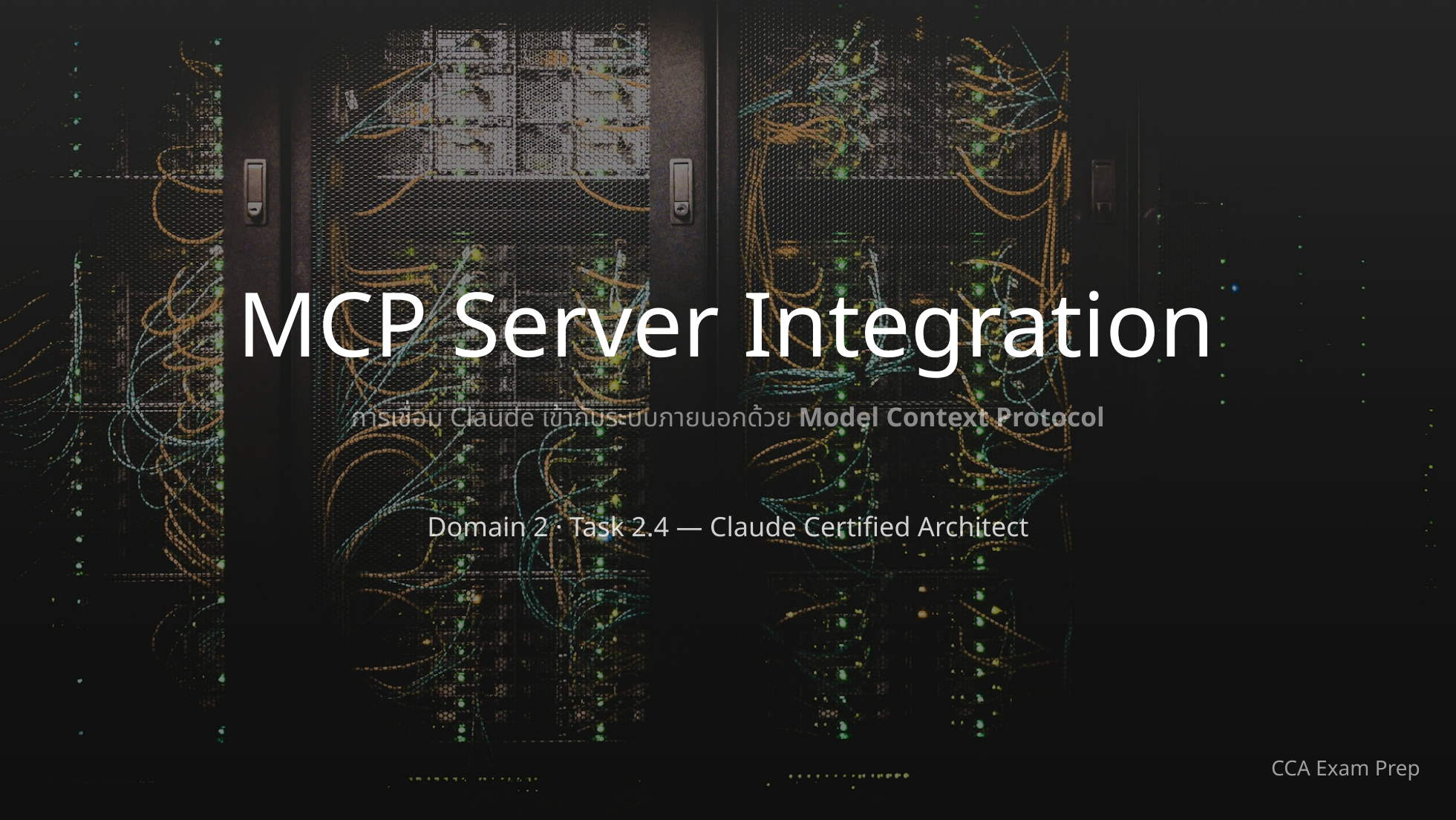




MCP Server Integration

การเชื่อม Claude เข้ากับระบบภายนอกด้วย **Model Context Protocol**

Domain 2 · Task 2.4 — Claude Certified Architect

MCP Server คืออะไร?

- **MCP (Model Context Protocol) server** คือตัวขยายความสามารถของ Claude โดยเชื่อมเข้ากับ**ระบบภายนอก**
- เช่น database · API · dev tools · issue tracker (Jira, GitHub, Slack)
- agent เรียกใช้ระบบเหล่านี้ผ่าน **tool** ที่ server จัดเตรียมให้

Task นี้**ไม่ได้**เน้นการเขียน protocol เอง — แต่เน้น "**การตัดสินใจเชิงสถาปัตยกรรม**" ว่าจะ config, scope และเลือกใช้ server อย่างไรให้ถูก

The Scoping Hierarchy

สอง scope ของการ config

Project-level — `.mcp.json`

- อยู่ที่ **root ของ repo**
- **version-controlled** และแชร์กับทั้งทีม
- เหมาะกับ server ที่ทุกคนต้องใช้ (Jira, GitHub, internal API)

User-level — `~/.claude.json`

- อยู่ที่ **home directory** ของผู้ใช้
- เป็น**ของส่วนตัว** ไม่ถูก version-control
- เหมาะกับ server ทดลอง หรือ integration ส่วนตัว



Key Concept: เลือก scope ให้ถูกที่

Project-level `.mcp.json`

version-controlled + แชร์ทั้งทีม

User-level `~/.claude.json`

ส่วนตัว ไม่แชร์

⚡ **ป้องกัน credential หลุด**

ใช้ `${ENV_VAR}` syntax เพื่อกันไม่ให้ credential หลุดเข้า version control

หลักคิด: config ที่**ทั้งทีมต้องใช้** → project-level · **ของทดลอง/ส่วนตัว** → user-level

tool ทั้งหมดถูก discover พร้อมกัน

★ จุดที่ข้อสอบชอบหยิบมาถาม

tool จาก**ทุก server** ที่ config ไว้ (ทั้ง project-level และ user-level) จะถูก **discover ตอน connection time** และพร้อมใช้งานพร้อมกันทั้งหมด

แปลว่า agent เห็น **tool รวมจากทั้งสอง scope** ในเซสชันเดียว
ไม่ได้แยกกันคนละโลก

Environment Variable Expansion

`.mcp.json` ใช้ syntax `${VARIABLE_NAME}` เพื่ออ้างถึง environment variable แทนการเขียน credential ตรงๆ ในไฟล์

นักพัฒนาแต่ละคนตั้ง token ของตัวเองไว้ในเครื่อง · ไฟล์ config อ้างแค่ชื่อ **variable**

✓ config ปลอดภัยพอที่จะ commit ได้

✓ แต่ละคน authenticate ด้วย credential ตัวเอง

✓ rotate token ได้โดยไม่ต้องแก้ config

✓ ไม่มี secret รั่วผ่านประวัติของ repo

MCP Resources

แคตตาล็อกที่ลด tool call เกินจำเป็น

Resources คือการเปิดเผย "**แคตตาล็อกเนื้อหา**" ให้ agent เห็นล่วงหน้า โดยไม่ต้องเสีย tool call ไปสำรวจเอง

รายการ Jira issue

สารบัญเอกสาร

database schema

 **ประโยชน์: ลด query ที่ไม่จำเป็น**

ถ้าไม่มี resources → agent อาจต้องเรียก `list_tables` แล้วตามด้วย `describe_table` ที่ละ table กว่าจะรู้โครงสร้าง

✓ resources ให้ข้อมูลนั้นตั้งแต่ต้น

Build-vs-Use Decision

★ ออกสอบบ่อย

✓ ใช้ community server เมื่อ

- มี integration มาตรฐานอยู่แล้ว (Jira, GitHub, Slack, Linear, Notion)
- ผ่านการทดสอบและได้ update ต่อเนื่อง
- เวลาและภาระ maintenance เป็นเรื่องสำคัญ



สร้าง custom server เฉพาะเมื่อ

- มี workflow เฉพาะทีมที่ community server รองรับไม่ได้
- ต้องมี business logic เฉพาะที่ขึ้น tool
- ต้องเชื่อมกับระบบภายในที่เป็น proprietary และไม่มี community server รองรับ

Enhancing MCP Tool Descriptions

ปัญหา

คำอธิบาย tool ที่สั้นเกินไป (**sparse**) ทำให้ agent เลี่ยงไปใช้ built-in tool ที่มี documentation ละเอียดกว่าแทน → MCP tool เลยถูกละเลย

✅ วิธีแก้

เขียน description ยาว **3-5 ประโยค** อธิบายให้ครบว่า tool ทำอะไร · คำนาค่าอะไร · ควรใช้เมื่อไร · เทียบกับ built-in alternative อย่างไร

× ห้วนเกินไป

"Searches code"

✓ ละเอียด

semantic search ที่ AST-aware และรวม context มาให้ด้วย



Exam Traps ที่ต้องระวัง

1. สร้าง custom server สำหรับงานมาตรฐาน

สร้าง custom MCP server สำหรับ integration มาตรฐานอย่าง Jira ที่ community server มีอยู่แล้ว

2. เอา config ระดับทีมไปใส่ user-level

ใส่ไว้ใน `~/.claude.json` ทำให้เพื่อนร่วมทีมไม่ได้ใช้

3. commit credential ลงไฟล์ตรง ๆ

แทนที่จะใช้ environment variable expansion (`${ENV_VAR}`)

4. ปลอญ tool description ให้ห้วน

ทำให้ agent เป็น MCP tool แล้วไปเลือก built-in tool แทน



Practice Scenario

ลองคิดก่อนดูเฉลย

โจทย์: ทีมต้องการ integrate Jira เข้ากับ workflow ของ Claude Code · นักพัฒนาคนหนึ่งเสนอให้ "สร้าง custom MCP server ขึ้นมาเอง" — **แนวทางที่ถูกต้องที่สุดคืออะไร?**

- A. สร้าง custom MCP server สำหรับ Jira ตามที่นักพัฒนาเสนอ เพราะยิ่ง custom ยิ่งควบคุมได้เต็มที่
- B. ใส่ config Jira ไว้ใน `~/.claude.json` (user-level) เพื่อให้ทั้งทีมใช้ร่วมกัน
- C. ข้าม MCP ไปเลย แล้วให้ agent เรียก Jira REST API ผ่าน Bash โดยตรง
- D. ประเมิน community MCP server สำหรับ Jira ที่มีอยู่ก่อน แล้วค่อยสร้าง custom เฉพาะกรณีที่มันรองรับ workflow เฉพาะทีมไม่ได้



หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option D

Jira เป็น integration มาตรฐานที่มี community server พร้อมใช้ ผ่านการทดสอบและได้ update ต่อเนื่อง — ให้ **ประเมินตัวที่มีอยู่ก่อน** แล้วสร้าง custom เฉพาะเมื่อมี workflow เฉพาะทีมที่มันทำไม่ได้จริง ๆ

A ผิด — ตกกับดัก "สร้าง custom สำหรับงานมาตรฐาน" เพิ่มภาระ maintenance โดยไม่จำเป็น

B ผิด — ผิด scope เพราะ `~/.claude.json` เป็น user-level ส่วนตัว ไม่แชร์ทั้งทีม (ของทีมต้องอยู่ใน project-level `.mcp.json`)

C ผิด — ทั้งประโยชน์ของ MCP (tool discovery, resources, description ที่ agent เข้าใจ) แล้วผลกระทบ auth และ error handling ไปไว้ที่ Bash แบบดิบ ๆ



Build Exercise (30 นาที)

ฝึกลงมือจริง 5 ขั้นตอน

- 1 สร้าง `.mcp.json` ที่ชี้ไป community server (กำหนด `command` และ `args`)
- 2 ใช้ `${GITHUB_TOKEN}` เป็น environment variable expansion แทนการฝัง token
- 3 เพิ่ม server ส่วนตัวไว้ใน `~/.claude.json`
- 4 เปิด content catalog เป็น MCP resource (เช่น database schema หรือ documentation)
- 5 ปรับ tool description ให้ละเอียดเพื่อกันไม่ให้ agent เลือก built-in tool แทน



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

ทีมต้องการ integrate กับ Jira เพื่อทำ issue tracking ใน workflow ของ Claude Code · นักพัฒนาเสนอให้สร้าง custom MCP server ขึ้นมาเอง — ขั้นตอนแรกที่ถูกต้องคืออะไร?

- A. สร้าง custom MCP server ที่มี API endpoints ตรงตามที่ทีมต้องการเลย
- B. ประเมิน community MCP server สำหรับ Jira ที่มีอยู่ก่อน แล้วค่อยสร้าง custom เฉพาะกรณีที่มันรองรับ workflow เฉพาะทีมไม่ได้
- C. เรียก Jira REST API ตรง ๆ ผ่าน Bash commands แทนการใช้ MCP
- D. ใส่ Jira integration ไว้ใน `~/.claude.json` เพื่อให้ นักพัฒนาแต่ละคน config เองแยกกัน

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

สำหรับ integration มาตรฐาน ควรเลือก community server เป็นตัวเลือกแรกเสมอ เพราะมีการ maintain ทดสอบ และครอบคลุม use case ทั่วไปแล้ว — สร้าง custom ก็ต่อเมื่อ community server รองรับการต้องการเฉพาะทีมไม่ได้จริง ๆ

ทำไมข้ออื่นผิด

A — สร้าง custom เร็วเกินไป ทั้งที่มี community MCP server สำหรับ Jira ที่ครอบคลุม use case มาตรฐานอยู่แล้ว

C — เรียก API ตรงข้าม MCP tool interface ไป ทำให้เสียประโยชน์ของ tool descriptions, structured responses และ agent-native integration

D — integration ของทั้งทีมควรอยู่ใน project-level `.mcp.json` เพื่อ version-control และแชร์ร่วมกัน ส่วน `~/.claude.json` ไว้สำหรับ server ส่วนตัว



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

นักพัฒนา commit คำนีลง `.mcp.json` : `{"env": {"GITHUB_TOKEN": "ghp_abc123xyz789"}}` — ปัญหา
ด้าน security คืออะไร?

A. format ของ token ไม่ถูกต้องสำหรับ GitHub

B. ค่า credential จริงถูก commit เข้า version control แทนที่จะใช้ environment variable expansion แบบ `${GITHUB_TOKEN}`

C. ควรย้าย token ไปไว้ใน `~/.claude.json` แทน `.mcp.json`

D. key `env` ไม่ใช่ field ที่ใช้ได้ ใน config ของ `.mcp.json`

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

credential ที่ commit ตรง ๆ ลง `.mcp.json` จะเข้าไปอยู่ใน history ของ repo และมองเห็นได้โดยทุกคนที่เข้าถึงได้ — วิธีที่ถูกคือใช้ syntax `${GITHUB_TOKEN}` แล้วให้นักพัฒนาแต่ละคนตั้ง token ของตัวเองในเครื่อง

ทำไมข้ออื่นผิด

- A — format ของ token ถูกต้องตาม syntax อยู่แล้ว (มี prefix `ghp_`) ปัญหาคือมันถูกเปิดเผย ไม่ใช่ malformed
- C — ย้ายไป `~/.claude.json` ก็ไม่ช่วย ถ้าค่าจริงยังถูก hard-code อยู่ ทางแก้คือ environment variable expansion ไม่ใช่เปลี่ยนไฟล์
- D — `env` เป็น field ที่ใช้ได้ปกติใน `.mcp.json` ปัญหาอยู่ที่ค่า ไม่ใช่ที่ key



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

agent เรียก tool ถึง 5 ครั้งเพียงพอเพื่อทำความเข้าใจโครงสร้าง database: `list_tables` ก่อน แล้วตามด้วย `describe_table` อีก 4 table — MCP feature ใดจะช่วยลด exploratory calls เหล่านี้?

- A. cache ผลลัพธ์ของ tool ไว้ เพื่อให้ request ครั้งถัดไปเร็วขึ้น
- B. เปิด database schema เป็น MCP resource เพื่อให้โครงสร้างพร้อมใช้โดยไม่ต้องเรียก tool
- C. เพิ่ม tool เดียวชื่อ `get_full_schema` ที่คืน description ของทุก table ในครั้งเดียว
- D. ใส่ schema ลงใน system prompt เพื่อให้ agent รู้อยู่แล้ว

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

MCP resource ใช้เปิด content catalogue (เช่น database schema) ให้ agent โดยไม่ต้องเรียก tool สํารวจ — schema พร้อมใช้ทันทีตั้งแต่ตอน connection จึงตัดการเรียก `list_tables` และ `describe_table` ออกไปได้

ทำไมข้ออื่นผิด

- A — cache ทำให้เรียกซ้ำเร็วขึ้น แต่ไม่ตัด exploratory calls ครั้งแรก agent ก็ยังเรียก 5 ครั้งอยู่ดี
- C — tool เดียวดีกว่า 5 tool แต่ก็ยังต้องเรียก tool อยู่ดี ขณะที่ MCP resource ให้ข้อมูลโดยไม่ต้องเรียกเลย
- D — system prompt ไม่เหมาะกับ structured data อย่าง schema เพราะกิน context token ทุก request และไม่ update เมื่อ schema เปลี่ยน



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

ทีมมี project-level servers ใน `.mcp.json` และนักพัฒนาคนหนึ่งมี personal servers ใน `~/.claude.json` — เมื่อ Claude Code connect แล้ว tool ไດบ้างที่ใช้ได้?

- A. เฉพาะ project-level tools จาก `.mcp.json` เพราะมี precedence สูงกว่า
- B. เฉพาะ user-level tools จาก `~/.claude.json` เพราะ config ส่วนตัว override project
- C. tool ทั้งหมดจากทั้ง project-level และ user-level servers ถูก discover ตอน connection และใช้ได้พร้อมกัน
- D. นักพัฒนาต้องเลือกเองว่าจะ activate server level ไหน

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option C

tool ทั้งหมดจากทุก server ที่ config ไว้ — ทั้ง project-level (`.mcp.json`) และ user-level (`~/.claude.json`)
— ถูก discover ตอน connection และใช้ได้พร้อมกัน ไม่มีขั้นตอน activate เอง และไม่มี level ใด override อีก level

ทำไมข้ออื่นผิด

- A — user-level servers ไม่ได้ถูกตัดออก ทั้งสอง level ต่างก็ contribute tool
- B — project-level servers ไม่ได้ถูก override ทั้งสอง level ต่างก็ contribute tool
- D — ไม่มีการ activate server เอง discovery เกิดขึ้นอัตโนมัติตอน connection



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

MCP tool ชื่อ `search_codebase` มี description ว่า "Searches code." แต่ agent เลือกใช้ built-in Grep tool แทนเสมอ — เพราะอะไร และแก้อย่างไร?

A. MCP tool ชำกว่า Grep agent เลยเลือกตาม performance

B. description ที่บางเบาของ MCP tool แพ้ built-in description ที่ละเอียดของ Grep — แก้โดยเสริม description ให้อธิบาย capabilities และ outputs อย่างละเอียด

C. built-in tools มี priority เหนือ MCP tools เสมอไม่ว่า description จะเป็นอย่างไร

D. ควร rename tool เป็น `grep_enhanced` เพื่อสื่อว่ามันมาแทน Grep

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

model จะชอบ tool ที่มันเข้าใจดีที่สุด — Grep มี built-in description ที่ละเอียด ขณะที่ description บางเบา ("Searches code") ให้ context ไม่พอที่จะให้ model เลือกใช้ · การเสริม description ให้อธิบาย capabilities, return format และควรใช้เมื่อไรเทียบกับ Grep จะแก้ปัญหานี้ได้

ทำไมข้ออื่นผิด

- A — agent ไม่มีข้อมูล performance metrics มันเลือก tool จาก description ไม่ใช่ความเร็ว
- C — built-in tools ไม่ได้มี priority ในตัว การเลือกขึ้นกับคุณภาพ description และความเกี่ยวข้องกับ query
- D — ตั้งชื่อ MCP tool ให้เหมือน built-in ยิ่งสร้างความสับสน ทางแก้คือ description ที่ดีขึ้น ไม่ใช่ชื่อที่ทำให้เข้าใจผิด